

Authentication

FastAPI Masterclass

Intro to Authentication

- **Authentication** is the act of logging the user into the system.
 - Is bob@example.com a real user? Did he provide the right password?
- **Authorization** is the act of allowing or prohibiting the user from taking an action.
 - Does Bob have permission to delete a record?

Token-Based Authentication

- Many modern APIs use a strategy of **token-based authentication**.
- There are other authentication strategies like sessions, OAuth, or API keys.
- A **token** is a string/credential that the client sends on every request to prove that they are authenticated.

The Flow I

- The client sends their credentials (i.e., email and password) to an endpoint.
- The server verifies the credentials, ensuring they match a user record in the database.
- If the credentials are valid, the server creates and sends back a token, a string that represents the user's identity. Every user will have a unique token.

The Flow II

- The client includes the token in every subsequent request to the server, typically as a header.
- The client chooses how to store the token on its end (cookies, localStorage, etc)
- The server parses the token on every request and verifies its legitimacy. The token identifies who is making the request.

The Flow III

- If no token is attached to a request, the server assumes the client is unauthenticated and asks them to log in again.
- Tokens expire over time. When a token expires, the server asks the client to log in again.

Statelessness

- Token-based authentication is stateless. There is no database row storing a user's "logged-in" state.
- The attachment of a token on subsequent requests after login is the client's *proof* they are logged in.
- HTTP is a stateless protocol so either (a) the server has to keep state on who is logged in or (b) the client has to prove they are logged in.

Why Tokens?

- Server does not need to query database to verify user is logged in on every request.
- Sending a password repeatedly over the network increases the chance of it being intercepted.
- Multiple backends/APIs can rely on the same token to authenticate the user across multiple services.

The Battle Plan

- We'll build an authentication system from scratch.
- We'll support a login endpoint that returns a JWT token.
- We'll protect access to routes so only authenticated users can access them.
- We'll define role-based access control (admins and regular users).